BTECH PROJECT • PROGRESS REPORT • OCTOBER 2026

Learning Which Edge to Add: GNN-based Link Prediction for Minimizing Average Betweenness Centrality

Code: https://github.com/snehanagmoti/gnn-avg-betweenness

Abstract. We study which single missing edge most reduces a network's *average betweenness centrality*, and train a GraphSAGE-based ranking model (adapted from Aashish et al.) to predict it without exhaustive evaluation. An exact ground-truth method makes data generation about 450× faster than brute force. On unseen Barabási–Albert graphs the model reaches mAP@10 = 0.65, outperforming four heuristic baselines (best: 0.49); improving its ranking head remains open.

1 Introduction

Betweenness centrality [5] measures how often a node lies on shortest paths between other nodes. In communication, transport and power networks it reflects *load*: high-betweenness nodes are congestion points and single points of failure. Adding a link creates alternative shortest routes and spreads this load.

We ask: **which single missing edge most reduces the average betweenness of a network?** A network has $O(n^2)$ candidates and each exact evaluation recomputes betweenness for the whole network, which already takes about 30 minutes for a 200-node graph. We therefore *learn* a model that ranks all candidates in one pass. The work adapts Aashish et al. [1], who solve this task for the *maximum* betweenness. Contributions so far:

- a modular, reproducible pipeline (dataset, model, training, evaluation), available at https://github.com/snehanagmoti/gnn-avg-betweenness;
- an exact, fast ground-truth computation for the average-betweenness objective;
- an adaptation of the paper's GNN ranking model (new labelling rule, sampling and tie handling);
- an evaluation on unseen graphs against four heuristic baselines.

2 Problem Definition

Let $G = (V, E)$ be a connected, undirected, unweighted graph with n nodes, and let $\bar{E}$ be the set of node pairs that are not yet connected (the *candidate edges*). The betweenness of a node v counts the fraction of shortest paths between every other pair s, t that pass through v; the *average betweenness* $\overline{\mathrm{BC}}(G)$ is its mean over all nodes. For a candidate $e \in \bar{E}$ we define its **reduction**

$$r(e) = \overline{\mathrm{BC}}(G) - \overline{\mathrm{BC}}(G + e), \tag{1}$$

and the goal is the best candidate $e^* = \arg\max_{e \in \bar{E}} r(e)$. Since the evaluation looks beyond the single best edge, the learning task is to produce a **ranking** of all candidates that places the edges with the largest reductions first. The base paper instead maximizes the drop in $\max_v \mathrm{BC}(v)$.

3 Pipeline Overview

The system is organised as four steps, each implemented as one script that reads the output of the previous step (Fig. 1). All settings are kept in a single configuration file, and the complete chain runs in about one to two minutes on a laptop CPU.

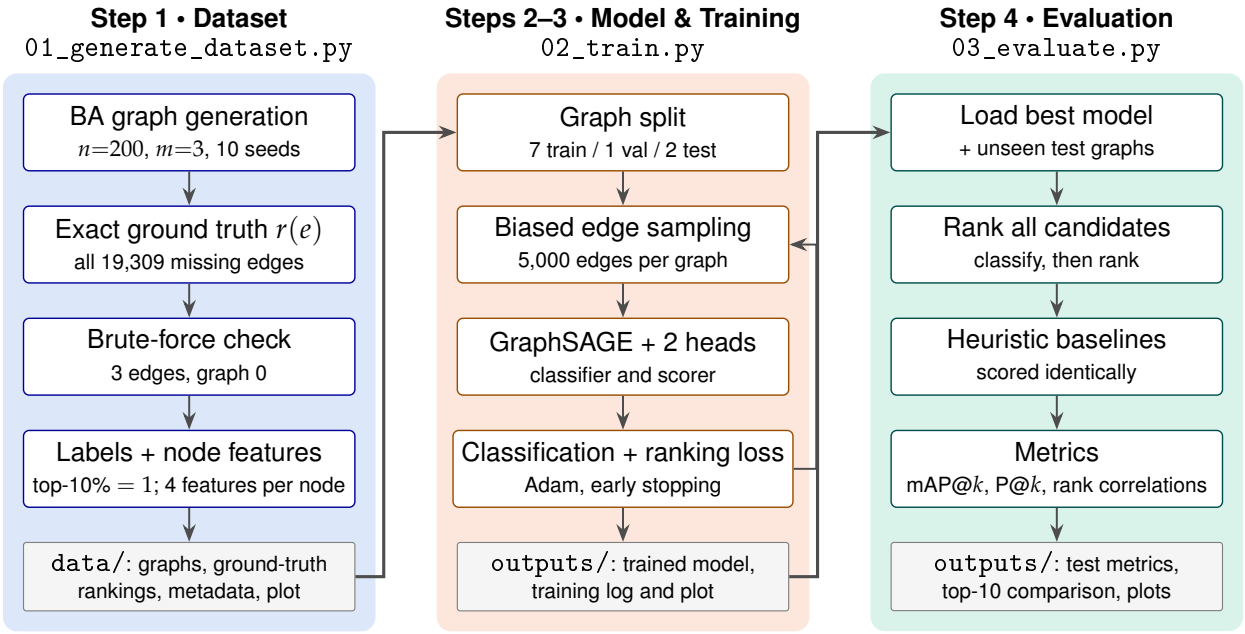


Figure 1. End-to-end pipeline. Each lane is one script; the loop in Steps 2–3 repeats once per epoch.

Step 1. Dataset. Generate synthetic networks, compute the exact reduction of every candidate edge, and derive labels and node features (Section 4).

Step 2. Model. A GNN that turns the network into node embeddings and scores every candidate edge (Section 5).

Step 3. Training. Fit the model on training graphs and select the best version on a validation graph (Section 5.3).

Step 4. Evaluation. Rank all candidates of unseen test graphs and compare with heuristic baselines (Sections 6–7).

4 Dataset Generation

Networks. We generate 10 Barabási–Albert (BA) graphs [4] with 200 nodes each, where every new node attaches to $m = 3$ existing nodes with probability proportional to their degree. This produces scale-free networks with a few large hubs and many low-degree nodes, similar to many real-world networks. Each graph has 591 edges and therefore 19,309 candidate edges. Graphs are generated with fixed seeds, so the dataset is exactly reproducible.

Exact ground truth. The ground truth is the reduction $r(e)$ for *every* candidate. Recomputing betweenness with Brandes' algorithm [3] for each candidate would take about 32 minutes per graph. We avoid this with a simple observation: on a connected, unweighted graph, every shortest path between s and t passes through exactly $d(s,t) - 1$ intermediate nodes, so the total betweenness of all nodes equals

$$\sum_{v} \mathrm{BC}(v) = \sum_{\{s,t\}} \big(d(s,t) - 1\big). \tag{2}$$

Average betweenness therefore depends only on pairwise distances; minimizing it is equivalent to minimizing the average shortest-path length. Adding an edge (u, v) can only shorten distances, and the new distance between any pair is the shorter of its old distance and a route that uses the new edge. Algorithm 1 exploits this: it computes all pairwise distances once and then evaluates each candidate by updating the distance matrix, which takes about 4 seconds per graph (roughly 450× faster) and gives exactly the same values. Every run re-checks three random candidates against the standard brute-force computation.

Algorithm 1: Exact ground truth for one graph

Input: connected graph G with n nodes
Output: reduction $r(e)$ for every candidate edge e
Compute the all-pairs shortest-path distance matrix D ; ▷ *once, Floyd–Warshall*
foreach *missing edge* (u, v) **do**
  $D' \leftarrow$ for every pair (s, t), the shorter of D_{st} and the two routes $s \to u \to v \to t$, $s \to v \to u \to t$;
  $r(u, v) \leftarrow$ total distance saved $\sum(D - D')$, halved for double counting and divided by n;
Rescale all r to NetworkX's normalized betweenness scale;
Verify three random candidates with brute-force betweenness ; ▷ *abort if any differs*

Every candidate strictly reduces average betweenness: no shortest path gets longer and the endpoints move from distance ≥ 2 to 1. Unlike the maximum-betweenness setting, no edge makes things worse.

Labels. Since all reductions are positive, the paper's implied rule ("the edge lowers the objective") would label every edge positive. We label an edge positive if it is in the top 10% of its graph, ties included (about 2,000 per graph).

Node features. Instead of the paper's random initialization, each node is described by four structural features that transfer to unseen graphs: log-degree, clustering coefficient, closeness and core number, standardized per graph. Closeness is directly tied to the objective, as it measures distance to all other nodes.

Ground-truth structure. Reductions have a heavy head and a long tail: the best edge is 6–11× the median, and most candidates help only marginally (Table 1, Figs. 2–3). The best edges are typically *shortcuts from a major hub to a peripheral node*. Only about 130–145 distinct reduction values exist among 19,309 candidates, so exact ties are frequent.

	Min	Max	Mean	Interpretation
Largest node degree	33	50	42.2	a few dominant hubs
Best reduction $r(e^*)$ ($\times 10^{-5}$)	3.60	5.30	4.33	normalized scale
Best / median reduction	6.3×	11.1×	8.1×	few edges matter a lot
Distinct reduction values	129	145	137.5	many exact ties
Top-10 edges touching the largest hub	2	9	5.0	hub-to-periphery shortcuts

Table 1. Ground-truth statistics over the 10 graphs ($n = 200$, $m = 3$, diameter 5).

5 Proposed Method

5.1 Why a graph neural network?

A good edge depends on the *global* position of both endpoints: how central u is, how peripheral v is, and how many shortest paths the link shortens. Hand-crafted per-node scores capture only

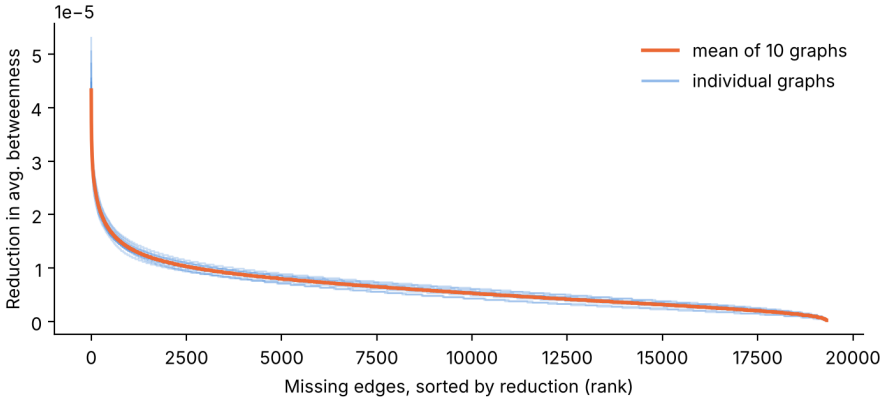
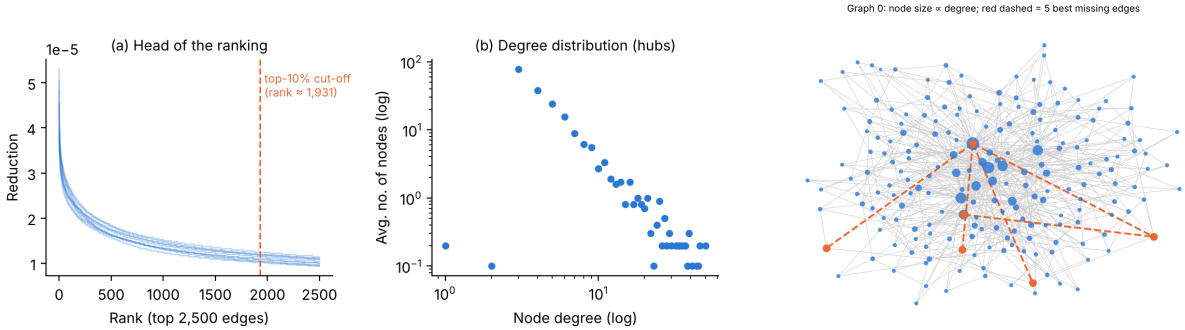


Figure 2. Reduction of every candidate edge, sorted in decreasing order, for each graph (blue) and the rank-wise mean (orange). The curve never falls below zero.



(a) The 2,500 best edges with the top-10% label cut-off (left); degree distribution (right).

(b) Graph 0 with its five best candidates (orange); four attach to the largest hub.

Figure 3. Structure of the ground truth.

part of this (best heuristic: mAP@10 = 0.49, Section 7). A GNN learns this automatically: each node repeatedly aggregates information from its neighbours, so its representation summarizes its multi-hop neighbourhood, and the trained model applies directly to unseen graphs.

5.2 Model architecture

The model (Fig. 4) has three parts.

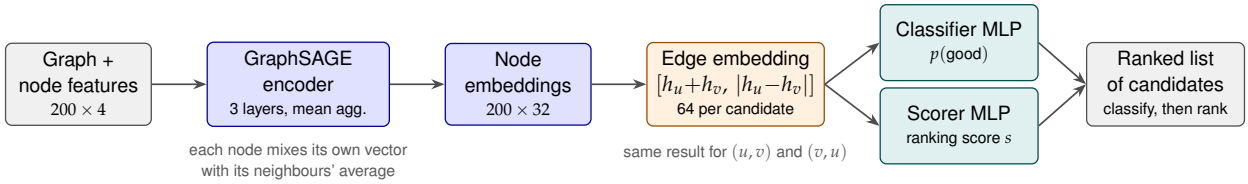


Figure 4. Model architecture. The encoder runs once per graph; the two heads then score any number of candidate edges.

Node encoder (GraphSAGE). As in the base paper we use GraphSAGE [2] with the mean aggregator. In each layer a node averages its neighbours' vectors, concatenates this with its own vector (keeping "who I am" separate from "what surrounds me") and applies a learned linear map. Three layers ($4 \to 32 \to 32 \to 32$, LeakyReLU, dropout) give each node a 32-dimensional embedding of its 3-hop neighbourhood, which covers most of a diameter-5 BA graph from a hub.

Edge representation. A candidate is represented by the sum and absolute difference of its endpoint embeddings. Unlike the paper's concatenation, this is independent of the order of u and v, and the difference term directly captures how dissimilar the endpoints are, matching the hub-to-periphery pattern.

Two prediction heads. Two MLPs (three hidden layers of 64 units) read the edge representation. The **classifier** predicts whether the edge is in the top 10%, acting as a coarse filter; the **scorer** outputs a score whose only role is to order edges. At inference, *classify-then-rank* places accepted edges first, each group ordered by score (Algorithm 2). This two-stage design, from the base paper, handles the strong imbalance: few of the 19,309 candidates matter. The model has about 29,600 parameters.

Algorithm 2: Ranking the candidates of a new graph (inference)

Input: graph G, trained model
Output: all candidate edges, ordered from best to worst

Compute node features and node embeddings with the encoder ; ▷ *one pass over G*
foreach *missing edge* (u, v) **do**
    Build the edge representation from the embeddings of u and v;
    Compute $p(\text{good})$ with the classifier and the score s with the scorer;
Place edges with $p(\text{good}) > 0.5$ first, ordered by s; append the remaining edges, ordered by s;

5.3 Training procedure

Data split. Splitting is done by *whole graphs*: graphs 0–6 are used for training, graph 7 for validation and graphs 8–9 for testing. The model therefore never sees any part of a test graph, which measures how well it generalizes to new networks.

Biased edge sampling. To avoid training on the uninformative long tail, each epoch draws 5,000 candidates per graph with probability falling steeply with true rank (the paper's x^4 idea, applied to the top only). This covers 88% of the top-100 edges and makes about one in three samples positive (vs. one in ten overall).

Losses. As in the paper, both heads are trained on the sum of a **cross-entropy loss** against the top-10% labels and a **triplet margin ranking loss** on random triplets of positive edges, which requires the edge with the larger true reduction to score higher by a margin of 1. Pairs with tied true reductions are skipped.

Optimisation. Adam updates the weights after each training graph. After every epoch the model is scored by mAP@10 on the validation graph; the best weights are kept and training stops after 50 epochs without improvement (Algorithm 3).

6 Experimental Setup

Evaluation protocol. The model ranks all 19,309 candidates of each test graph; for a cut-off k, the *relevant* edges are the true top-k (ties included). The main metric, as in the paper, is

$$\text{AP@}k = \frac{1}{k} \sum_{j=1}^{k} \text{P@}j \cdot \text{rel}(j), \tag{3}$$

where P@j is the precision of the model's first j edges and $\text{rel}(j)$ marks whether its j-th edge is relevant; mAP@k averages over test graphs and rewards relevant edges ranked early. We also report Precision@k and Pearson, Spearman and Kendall correlations over the true top-k edges.

Algorithm 3: Training

Input: training graphs, validation graph
Output: model with the best validation mAP@10

Initialize the model randomly; best ← −1;
for *epoch* = 1 **to** 225 **do**
  foreach *training graph, in random order* **do**
    Sample 5,000 candidate edges, biased towards the best ones;
    Run the model on the graph and the sampled edges;
    Classification loss ← cross-entropy against the top-10% labels;
    Ranking loss ← triplet margin loss on triplets of sampled positive edges (ties ignored);
    Update all weights with Adam on the sum of both losses;
  Rank all candidates of the validation graph (Algorithm 2) and compute mAP@10;
  if *mAP@10* > *best* **then** store a copy of the weights; best ← mAP@10;
  if *no improvement in the last 50 epochs* **then** stop;
Restore the stored weights;

Baselines. The paper has none; we add four training-free heuristics: *random*, endpoint *distance*, *degree product*, and *closeness gap* × *distance* (central-to-peripheral links).

Hyper-parameters. See Table 2. Dropout and learning rate are lower than in the paper, whose values made the scorer collapse with only seven training graphs.

Setting	Value	Setting	Value
Graphs (train / val / test)	7 / 1 / 2	Edges sampled per graph	5,000
GraphSAGE layers / width	3 / 32	Triplets per sampled edge	10
MLP hidden layers / width	3 / 64	Ranking margin	1
Activation / dropout	LeakyReLU / 0.2	Optimizer / learning rate	Adam / 0.003
Positive label	top 10%	Max epochs / patience	225 / 50

Table 2. Hyper-parameters (all set in `config.py`).

7 Results and Discussion

Training behaviour. Validation mAP@10 peaked at **0.78 (epoch 21)**; training stopped at epoch 71 (Fig. 5). The classification loss fell from 0.67 to 0.34, but the ranking loss stayed at the margin (≈1.0): the scorer has not yet learned a fine ordering.

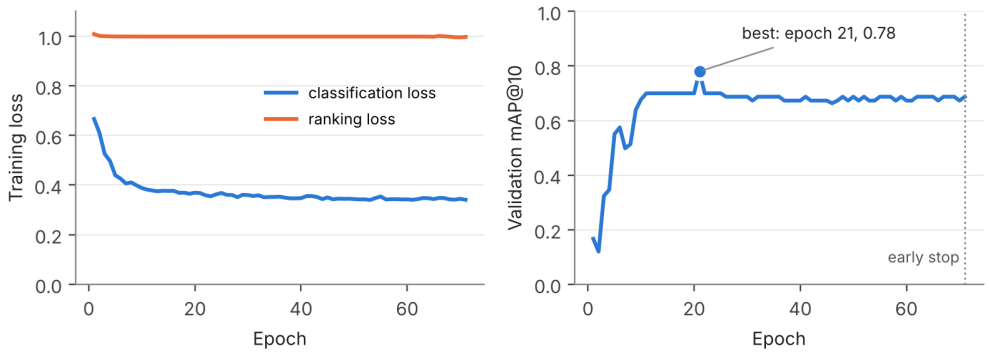


Figure 5. Training losses (left) and validation mAP@10 (right).

Test performance. On the two unseen test graphs (Table 3, Fig. 6) the GNN reaches mAP@10 $= 0.65$ and P@10 $= 0.70$ (7 of the true top 10 in each graph), finds the best edge in one graph, and beats all baselines for $k \geq 10$. Distance and degree product fail, since the farthest pairs are both peripheral and hubs are already close. The hub-to-periphery heuristic is strongest (mAP@10 $= 0.49$) and orders the top 10–20 slightly better (Spearman@10: 0.63 vs. 0.60), but degrades for larger k.

Method	mAP@1	mAP@10	mAP@50	mAP@100	P@10	Spearman@100	Kendall@100
GNN (ours)	**0.50**	**0.65**	**0.55**	**0.58**	**0.70**	**0.53**	**0.39**
Closeness gap × distance	**0.50**	0.49	0.36	0.43	0.50	0.33	0.24
Degree product	0.00	0.01	0.00	0.02	0.05	0.16	0.11
Distance	0.00	0.00	0.00	0.00	0.00	0.06	0.04
Random	0.00	0.00	0.00	0.00	0.00	0.07	0.04

Table 3. Results on the test graphs (19,309 candidates each).

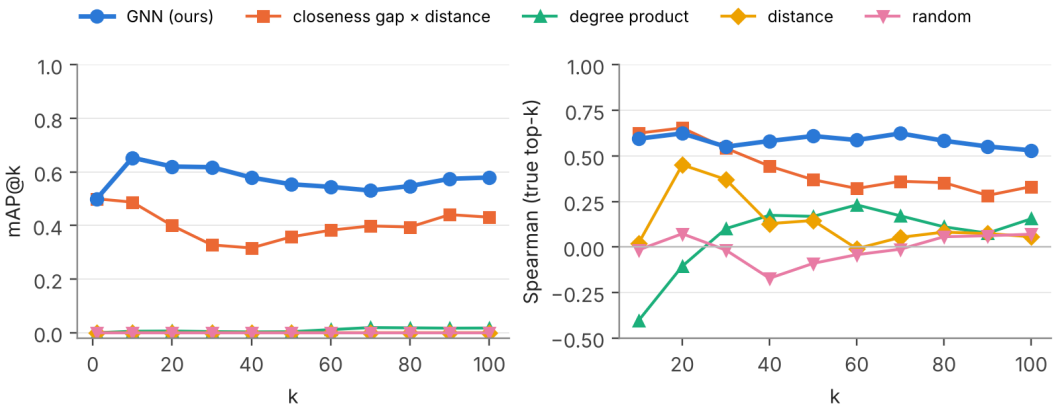


Figure 6. mAP@k (left) and Spearman correlation over the true top-k edges (right) on the test graphs.

Case study. In test graph 9 the model's top five equal the true top five; in graph 8 all ten predictions link the largest hub to peripheral nodes, missing three good edges through another node. The model has learned the dominant hub-to-periphery pattern but not finer distinctions, consistent with the weak ranking loss.

8 Comparison with the Base Paper

Aspect	This project	Aashish et al.	Same
Objective	Reduce average betweenness	Reduce maximum betweenness	no
Data	10 BA graphs, n=200, m=3	100 BA, 100 ER, 30 tree, 10 path; n 165–200, m 3–6	partly
Ground truth	Exact distance update (∼450× faster), verified	Brandes' algorithm per candidate	no
Labels	Top 10% of each graph	Edges that lower the objective (rule not stated)	idea only
Node features	Four structural features	Random initialization	no
Encoder	GraphSAGE, mean, 3 layers, width 32	GraphSAGE, mean, diameter−1 layers, width 32	mostly
Edge representation	Symmetric sum and difference, 64-d	Concatenation, 64-d	no
Heads and losses	Classifier + scorer; cross-entropy + triplet margin	same	yes
Ties in ranking loss	Skipped	Not handled	added
Sampling	5,000 edges, biased to the top	5,000 edges, biased to both extremes	partly
Optimisation	Adam, lr 0.003, dropout 0.2	Adam, lr 0.0085, dropout 0.5	partly
Evaluation	mAP@k, P@k, correlations, 4 baselines	mAP@k, correlations, no baselines	extended

Table 4. Design comparison with the base paper.

9 Limitations and Open Questions

Q1. Label rule. Is the top-10% rule appropriate? It labels about 1,930 edges per graph as positive, while the evaluation focuses on the top 10–100. A smaller fraction (e.g. 1%), or removing the classifier in favour of pure ranking or direct regression of the reduction, may suit this objective better.

Q2. Ties. With only about 140 distinct reduction values per graph, how should ties in the true top-k be treated in training and evaluation?

Q3. Dataset scope. Should we use $m = 3$ only or also 4–6, and add ER, tree and path graphs and mixed sizes when scaling up?

Q4. Paper fidelity. Should the model also be run with the paper's random features, concatenated edge representation and four layers (diameter − 1) for a like-for-like comparison?

Q5. Positioning. Minimizing average betweenness is equivalent to minimizing the average shortest-path length; should the work be related to existing results on that problem?

10 Conclusion and Next Steps

Our pipeline ranks candidate edges by their reduction of average betweenness; exact ground truth is about 450× faster than brute force, and the GNN beats all heuristics on unseen graphs. The ranking head is the main weakness. Next steps: resolve the open questions, improve the ranking head, and scale to more graphs, m values and graph types with repeated runs.

Code availability. Code, data, trained model and figures: `https://github.com/snehanagmoti/gnn-avg-betweenness` (run `python run_all.py`).

References

[1] Aashish, P. Diwakar, V. N. Kotyada, R. R. Singh, S. Pramanik. Deep Learning based Link Prediction for Minimizing Maximum Betweenness.

[2] W. L. Hamilton, R. Ying, J. Leskovec. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.

[3] U. Brandes. A Faster Algorithm for Betweenness Centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.

[4] A.-L. Barabási, R. Albert. Emergence of Scaling in Random Networks. *Science*, 286(5439):509–512, 1999.

[5] L. C. Freeman. A Set of Measures of Centrality Based on Betweenness. *Sociometry*, 40(1):35–41, 1977.